

LAS-NoCode Rule Editor: A visual low-code framework for reproducible Complex Event Processing rule design in IoT research

Rosiberto Gonçalves¹✉

¹ Estácio University Center Recife, Brazil ✉ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) ✉
- [Repository](#) ✉
- [Archive](#) ✉

Editor: [Open Journals](#) ✉

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Complex Event Processing (CEP) supports real-time detection of patterns and anomalies over continuous event streams and is widely adopted in Internet of Things (IoT), smart environments, and distributed systems. Despite mature execution engines, rule definition typically depends on textual domain-specific languages such as Event Processing Language (EPL), which increases cognitive load and introduces variability in rule specification.

LAS-NoCode Rule Editor (LAS) is an open-source, containerized, low-code web application that enables structured visual construction of CEP rules. LAS represents rules as directed block graphs and automatically generates syntactically valid EPL compatible with FIWARE Perseo CEP. The system is designed to support reproducible experimentation, structured rule modeling, and educational adoption in event-driven systems research.

Statement of need

CEP engines provide scalable and efficient runtime infrastructures, yet rule authoring remains largely manual and text-based. This creates challenges for:

- Teaching event-driven architectures and CEP concepts.
- Rapid prototyping of event patterns in research scenarios.
- Reproducible specification of rule logic across experiments.

In research contexts, rule definitions are often embedded in configuration files or scripts without structured modeling support, making controlled experimentation and replication more difficult.

LAS addresses this limitation by introducing a visual abstraction layer aligned with CEP semantics. The system:

- Encodes rule logic as structured block-based flows.
- Enforces consistent rule composition patterns.
- Automatically translates visual models into EPL.
- Enables containerized deployment for reproducible environments.
- Integrates natively with FIWARE Perseo CEP infrastructures.

Rather than replacing existing CEP engines, LAS formalizes the rule authoring layer, improving accessibility and reproducibility while preserving compatibility with production-grade infrastructures.

LAS has been used in practical IoT experimentation scenarios as part of a broader event-driven platform, where it supports visual specification and deployment of CEP rules.

State of the field

CEP has been a recognized research and industrial field since the formalization of event pattern detection models (Luckham, 2002). Engines such as Esper (EsperTech Inc., 2024) provide expressive EPL-based rule definition and efficient in-memory execution. Distributed stream processing frameworks including Apache Flink (Carbone et al., 2015) and Kafka Streams (Kreps et al., 2011) extend event processing to large-scale, fault-tolerant environments.

Within the FIWARE ecosystem, Perseo CEP (FIWARE Foundation, 2024) offers rule-based event processing integrated with context management services. However, these systems emphasize runtime execution and scalability rather than structured rule modeling or visual abstraction.

Visual flow-based tools such as Node-RED (Blackstock & Lea, 2014) support event-driven composition but are not specifically designed for EPL generation or formal CEP rule modeling. Unlike general-purpose flow editors, LAS enforces CEP-specific rule structure constraints aligned with EPL semantics, reducing syntactic and structural inconsistencies.

LAS contributes at the intersection of CEP and low-code tooling by:

1. Providing a domain-specific visual modeling environment tailored to EPL rule construction.
2. Automating transformation from visual graph structures to executable EPL.
3. Supporting reproducible experimental workflows via containerized deployment using Docker (Docker Inc., 2024).

By focusing on the rule authoring layer, LAS complements existing CEP infrastructures and addresses a gap between formal CEP engines and general-purpose visual workflow systems.

Software design

LAS is implemented as a modular web application composed of:

- Backend: Python with Flask (Pallets Projects, 2024).
- Frontend: JavaScript-based block editor built on Drawflow (jerosoler, 2024).
- Deployment: Docker-based containerization (Docker Inc., 2024).

Rules are internally represented as directed block graphs. The backend parses the graph into an intermediate representation and generates EPL statements. Generated rules can be deployed directly to FIWARE Perseo CEP via REST integration.

The system can operate as a standalone web application or as a Flask Blueprint embedded into larger systems. Containerization ensures consistent execution across development, experimentation, and teaching environments, reinforcing reproducibility.

Acknowledgements

The author acknowledges the open-source communities behind Flask, Docker, Drawflow, and FIWARE.

References

- Blackstock, M., & Lea, R. (2014). Node-RED: Low-code programming for event-driven applications. *Proceedings of the 5th International Workshop on the Web of Things*.
- Carbone, P., Katsifodimos, A., Ewen, S., & Markl, V. (2015). Apache flink: Stream and batch processing in a single engine. *IEEE Data Engineering Bulletin*, 38(4), 28–38.

- 78 Docker Inc. (2024). *Docker: Lightweight linux containers for consistent development and*
79 *deployment*. <https://www.docker.com/>
- 80 EsperTech Inc. (2024). *Esper complex event processing engine*. [https://www.espertech.com/](https://www.espertech.com/esper/)
81 [esper/](https://www.espertech.com/esper/)
- 82 FIWARE Foundation. (2024). *FIWARE perseo CEP*. [https://github.com/telefonicaid/perseo-](https://github.com/telefonicaid/perseo-core)
83 [core](https://github.com/telefonicaid/perseo-core)
- 84 jerosoler. (2024). *Drawflow: Simple flow library*. <https://github.com/jerosoler/Drawflow>
- 85 Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A distributed messaging system for log
86 processing. *NetDB*.
- 87 Luckham, D. (2002). *The power of events: An introduction to complex event processing*.
88 *Addison-Wesley*.
- 89 Pallets Projects. (2024). *Flask web development framework*. <https://flask.palletsprojects.com/>

DRAFT